

Use Cases and the Ever-Unfolding Story



www.netobjectives.com
Douglas Shimp
Doug_shimp@netobjectives.com

My Goals



Enlighten



Teach



Inspire

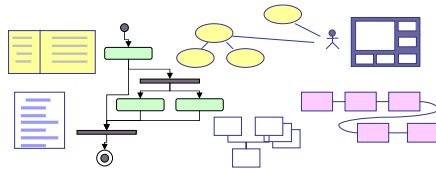
Purpose of Seminar

- Introduce you to Requirements Analysis as a discipline
- Describe an “effective” Use Case specifying *Functional Requirements*
- Show how a Use Case can be *unfolded* into high-level *Software Requirements*

Agenda

- A Look At Our Past
- Fundamental Problems
- What is Requirements Analysis?
- Use Case Discussion
 - What's the Scope?
 - Who Wants What?
 - How Do They Get it?
 - What Else Could Happen?
 - How Do We Handle That?
- Ever-Unfolding Story
- The Requirements Model/Document

A LOOK AT OUR PAST



The Purpose of a Project is...

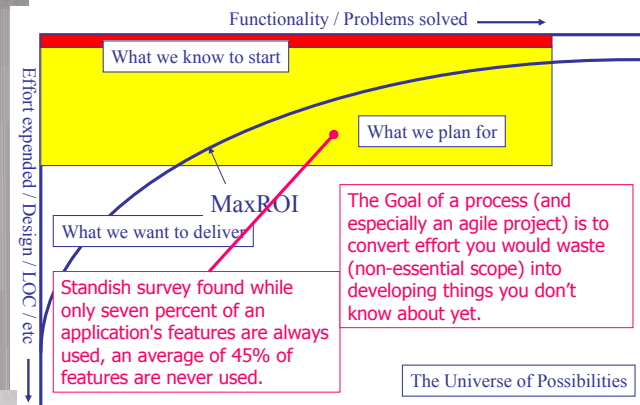
To provide a suitable solution for users...
That consists of quality code...
And doesn't cost too much



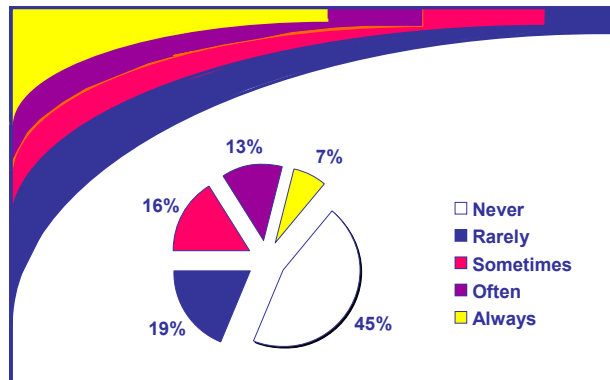
Where has our Industry been?

- How many of our projects have failed?
- How many of our projects succeeded?
- Is it getting any easier?
- Traditionally, we've been doing waterfallish development
 - The Standish Group says we fail at that...
- So, if we become agile
 - Will it make a difference?

Delivering Functionality in an Agile Project



Standish Study and “Ideal” Development

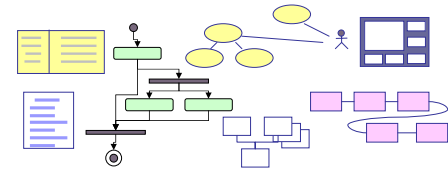


©Net Objectives, 3/25/2004

9

NETOBJECTIVES

WHAT IS REQUIREMENTS ANALYSIS?



©Net Objectives, 3/25/2004

10

NETOBJECTIVES

The Goals of Requirements Analysis

- To capture and refine requirements and document them in a Requirements Model
- To give people doing Production and Maintenance a “fighting chance” to get it right...



©Net Objectives, 3/25/2004

11

NETOBJECTIVES

Two Fundamental Facts about Requirements

1. They Change
 - Understanding of the problem changes as time passes
 2. They're never detailed enough
 - People want problems solved...
 - Developers need to write code...
 - There's a big delta here
- There's a natural tension here...
- When you have detail you're resistant to change
 - If you think it might change you're resistant to gather detail

©Net Objectives, 3/25/2004

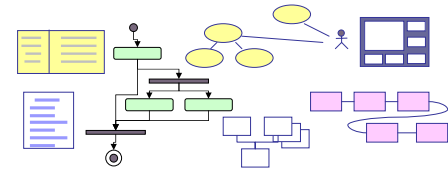
12

NETOBJECTIVES

What We Need Is...

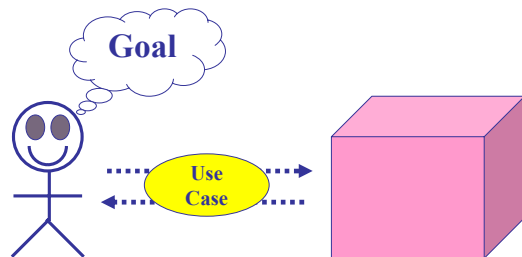
- A way to “flesh out” requirements (provide detail) that also allows change to occur
- This problem may be on the “too hard” pile
- What we can do is
 - Flesh out requirements a little at a time, so we only expand those we’re certain won’t change
 - Provide traceability and connectivity so that when we’re wrong we can see what else might change
- The tool we use for this is called the “Ever Unfolding Story”

USE CASE DISCUSSION



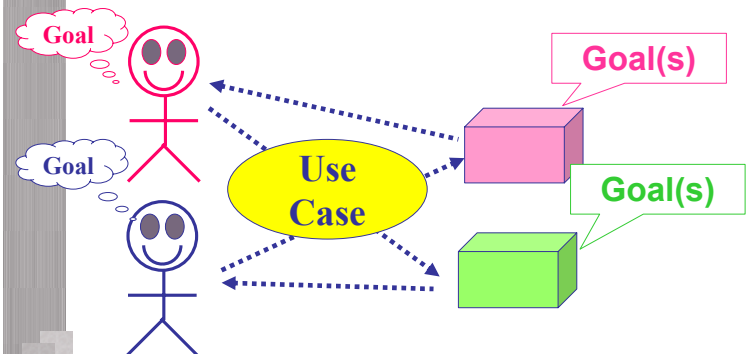
Use Case

- A Use Case describes a conversation between Actor(s) and System(s) in order to achieve Goal(s)



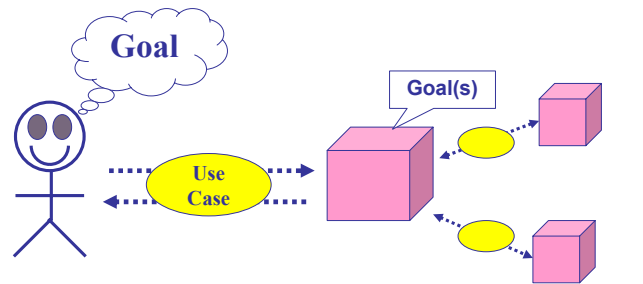
Sometimes

- This conversation is complicated



Sometimes

- This conversation is layered, and involves (sub)use cases



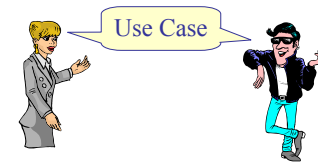
©Net Objectives, 3/25/2004

17

NETOBJECTIVES

Why the Use Case is Important

- We deliver solutions, not code
 - So we need to focus on solutions, not code
 - The use case describes how our product provides a solution
 - The use case is a good way to communicate with non-technical types (users...) about what the system will do for them
- Use Cases are large-grained
 - Organize many scenarios/stories that are part of the same goal
 - Allow everybody to think at the goal, rather than detail, level



©Net Objectives, 3/25/2004

18

NETOBJECTIVES

Example: Mary Gets "Fast Cash"

Mary, taking her two daughters to the day care center on the way to work, drives up to the ATM, runs her card across the card reader, enters her PIN code, selects FAST CASH, and enters \$35 as the amount. The ATM issues a \$20 and three \$5 bills, plus a receipt showing her account balance after the \$35 is debited. The ATM resets its screens after each transaction with FAST CASH, so Mary can drive away and not worry that the next driver will have access to her account. Mary likes FAST CASH because it avoids the many questions that slow down the interaction. She comes to this particular ATM because it issues \$5 bills, which she uses to pay the day care provider, and she doesn't have to get out of her car to use it.

This use case is a very casual one, just a story, really. We often use more ceremony in our use cases, and like the ones found in Cockburn's "Writing Effective Use Cases" book.

©Net Objectives, 3/25/2004

19

NETOBJECTIVES

Where the Use Case Fits

- A system level Use Case describes how the System Under Development (SUD) solves the User's problem
 - Documents "intentions" of the system
 - Is used to validate these "intentions" with the users



©Net Objectives, 3/25/2004

20

NETOBJECTIVES

An Effective Use Case is...

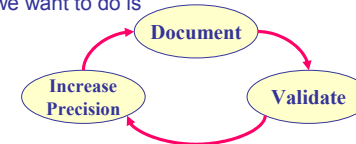
- Accurate and "Precise Enough"
- Well-Written
- The Focus of all Requirements
- Developed Iteratively
- Unfolded as we go...

Accuracy vs Precision

Definitions

- Accuracy: Validated, doing what is required
- Precision: Detailed, getting closer to development
- $\pi = 4.345687987989\dots$ is precise, but inaccurate...
- π is "about" 3 is accurate, but imprecise...

- What we want to do is



- We Define Five Levels of Precision

- What's the scope?
- Who Wants What?
- How Do They Get It?
- What Else Could Happen?
- How Do We Deal With That?

Text (written) Form

- Sequence of steps that progress the action
- Action Verbs, Short, punchy sentences

Login To ATM

Scope: ATM Machine

Goal Level: Sub-Function

Actor: ATM Customer

After swiping the ATM Card, the user is asked for a password. The system verifies that the card is legitimate and that the password corresponds to the card. The user is then given access to all the other ATM commands. The user is given three chances to enter a correct password; after the third time the ATM Card is kept by the machine.

Example: Withdraw Cash from ATM

Context of Use: ATM Customer wishes to withdraw money from ATM while executing the *Use ATM Machine* Use Case

Scope: ATM Machine

Goal Level: User-Goal

Primary Actor: ATM Customer

Precondition: ATM Customer is logged on

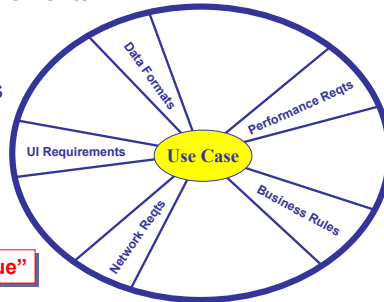
Main Success Scenario:

1. The ATM Customer selects an account to withdraw from, and enters an amount to withdraw, in multiples of \$20.
2. The ATM Machine verifies that there is enough money to dispense.
3. The ATM Machine notifies the Main Banking System of the Customer Account and amount being withdrawn, and the Main Banking System returns the new balance and (if appropriate) the fee amount.
4. The ATM Machine dispenses the cash and updates the receipt.

*The ATM Machine logs each step of this transaction.

Requirements Include

- Functional requirements
- Non-functional requirements
- Quality Measures
- User interface requirements
- Business rules
- Data integrity rules
- Environmental rules
- Network Protocols
- Etc



The Use Case is the "Glue"

Ever Unfolding Story

■ Use Case: Use Order Processing System

■ Header Info: ...

■ Main Success Scenario

1. User logs in and sets Access Permissions using Login
2. Repeat steps 3-4 until User selects Exit
3. System Displays Main Menu based on Access Permissions
4. User selects and runs one of the Main Functions (Place Order, Return Product, ...)

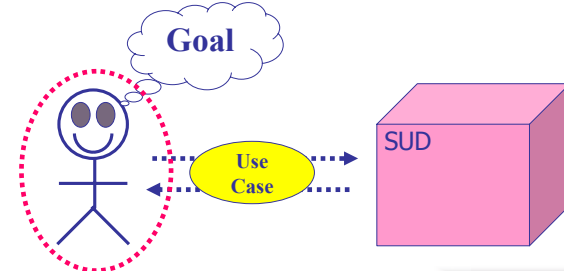
What's the scope?

Zeroth Level of Precision



Primary Actor

- Calls on the system to deliver one of its services
- The Primary Actor has the main goal to be satisfied – only one of them...
- Often initiates the action



Actors and Goals

■ An Actor is the entity (person) that actually interacts with the System under Discussion (SuD)

- An Actor's goals concern the "totality" of the use case
- Benefit the Actor gets from successful execution of use case
- An Actor can have Goals at various levels

■ For ATM Machine:

Actor	Goal
Customer	Withdraw Cash
	Deposit Check
	Transfer Funds
	Go To Dinner and a Movie (high level goal)
Teller	Refresh Cash Drawer
	Make Reports
Service Man	Replenish Receipt Paper
	Maintain Machine

Goal Levels

■ You are developing software for the SUD. You are collecting use cases that tell you what the functionality should be. These use cases come in three levels:

- Contextual (Business, Summary, Kite, Cloud) – provides context for the basic use cases
- System (User, Sea Level) – one-session interactions with the SUD, "complete" goal
- Internal (Subfunction, Fish, Clam) – incomplete, part of a System-level use case

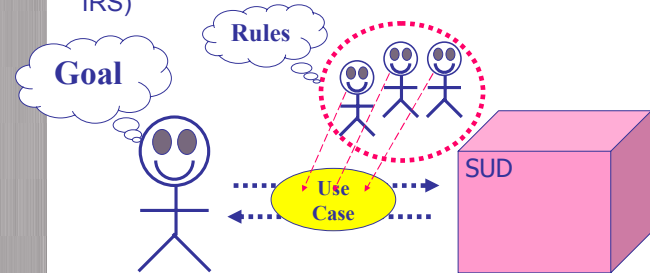
Who Wants What?

First Level of Precision



Stakeholder

- Somebody/thing that cares how the use case is executed – provides constraints and rules
- Not all stakeholders interact with the system (eg. The IRS)



Stakeholders and Interests

- A Stakeholder is someone or something that has an interest in the behavior of a use case
 - A Stakeholder's interests are usually captured in Business rules or Additional Goals
 - These interests usually concern the execution of the use case, not the "totality" of the use case
 - Often found / discovered late in the process...
- For ATM Machine

Stakeholder	Interest
Bank Owner	Assure Customer Properly Identified
	Don't inconvenience Customers too much
	Collect any (and all) applicable fees
FDIC	Follow All Banking Regulations
	Completely Track all Transactions

How Do They Get It?

Second Level of Precision



The Main Success Scenario - Definition

- A top-to-bottom description of one easy-to-understand and fairly typical scenario in which the goals are delivered and the stakeholder's interests are satisfied
- The "Sunny Day" scenario – showing that the problem can be solved, and giving one way to do it
- Often seen as the "guts" of the use case – the initial design of how the system will satisfy its responsibilities

MSS as a Series of Steps

- Each Step is one of:
 - An Interaction
 - A Validation
 - An Internal Change
- Example: Withdraw Cash
 1. The ATM Customer selects an account to withdraw from, and enters an amount to withdraw, in multiples of \$20.
 2. The ATM Machine verifies that there is enough money to dispense.
 3. The ATM Machine notifies the Main Banking System of the Customer Account and amount being withdrawn
 4. The Main Banking System returns the new balance and (if appropriate) the fee amount.
 5. The ATM Machine dispenses the cash and updates the receipt.
 - * The ATM Machine logs each step of this transaction.

Some Guidelines for the MSS

- Use Simple Grammar
- Show Clearly “who has the ball”
- Write from a Bird’s Eye View
- Show the Process Moving Forward
- Show the Intent, not the Movements
- Include a “reasonable” set of actions
- Validate, don’t “check whether”
- Optionally mention the timing

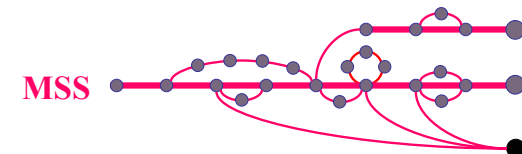
What Else Could Happen? (What Could Go Wrong?)

Third Level of Precision



Alternatives

- Use Cases contain all the alternatives to attain a given goal, including failure alternatives
- In this formulation of use cases, each of these alternatives is based on an extension off of the MSS



Identify Extension Conditions

- For Each Step in the MSS:
 - Brainstorm all Conceivable Failures and Alternative Courses
- Be Sure to Consider:
 - Alternative Success Paths
 - The Actor behaving incorrectly
 - Validation failing
 - Internal Failures in System
 - Expected failure modes
 - Abnormal failure modes
 - Critical Performance Failures

Say You Need to Handle Names

- Names are how people identify themselves
 - John A. Smith
 - Prof. Frederick Martin Biedenweg III, PE
 - Chuck O'Reilly, Susan Walton-Smythe
 - Albert van der Hoeven
 - Frank O. Adolph Wolfeschlegelsteinhausenbergerdorft
 - Cher, Madonna, Sting
 - Nguyen Bich → Frank Bich Nguyen → Frank Bich
 - Tchang Kai Chek, Chang Kai Chek, Chiang Kai-shek, Jiang Jieshi, 蒋介石
 - ㄅㄨ (the artist formerly known as Prince)
- As you can imagine, there is a large cost difference between handling the basic functionality and handling it all

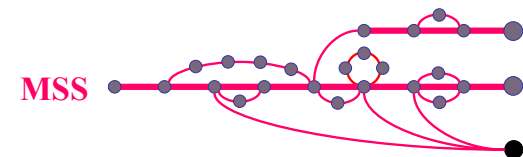
How Do We Deal With That?

Fourth Level of Precision



The Meat of the System

- The hardest part about any system is exception handling
- This use case formulation brings this problem right out in the open
- The users (or their surrogates) get to chime in on what should be done in response to various conditions
- If you don't have it in the use case, then you're letting the developer decide...



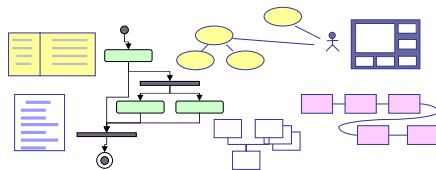
So, what happens?

- For each Extension Condition, you can:
 - Clean Up Mess and Bail Out (CUMABO)
 - Interact with the user for a few extra steps and reenter the MSS
 - Fork to a different Use Case
- Deciding which Extension Conditions to handle, and how, is one of the biggest problems you will have
 - You don't know them all up front, so they always hit you unexpectedly
 - They take resources to solve...

For Example...

- Say you have Extension
 - 2a. Insufficient Funds in ATM
- Then do you:
 - Terminate the user and Shut Down the ATM?
 - Terminate the user and wait to see if you can satisfy the next guy?
 - Give as much money as you can?
 - Give the user the option to choose a smaller amount?
- These options have different costs and benefits, and what to do needs to be decided by the right people...

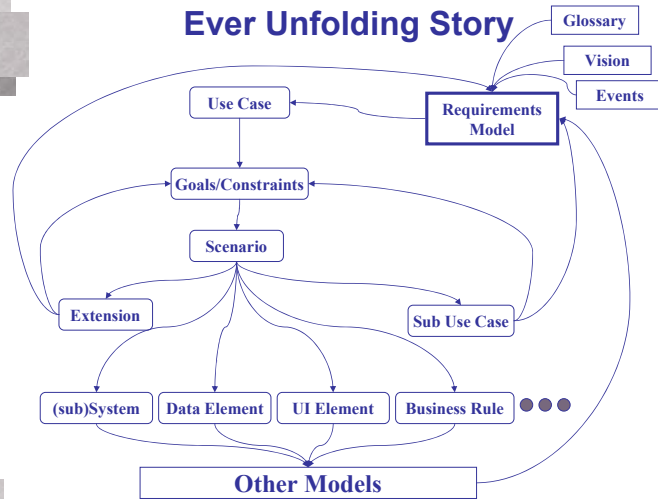
EVER UNFOLDING STORY



Use Case Based Requirements Analysis

- Now, all requirements analysis is based on an “unfolding story” of some sort
- Our version is centered on Use Cases
- Conceptually, we work from the “outside in”
 - What is the user's experience?
 - What does this mean for our interfaces?
 - What does this mean for our data?
 - What business rules are there?
- Our unfolding is tightly coupled to increasing the precision levels of use cases...

Ever Unfolding Story

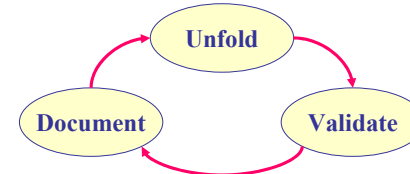


As it Unfolds (Accuracy vs Precision)

Just like with the use cases, we want to...

- Increase precision
- Validate
- Document
- Repeat...

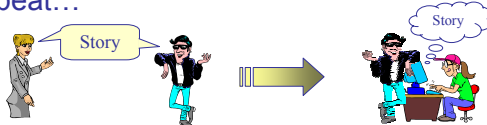
What we want to do is



We increase the precision of the use cases, and unfold the story...

Basic Strategy

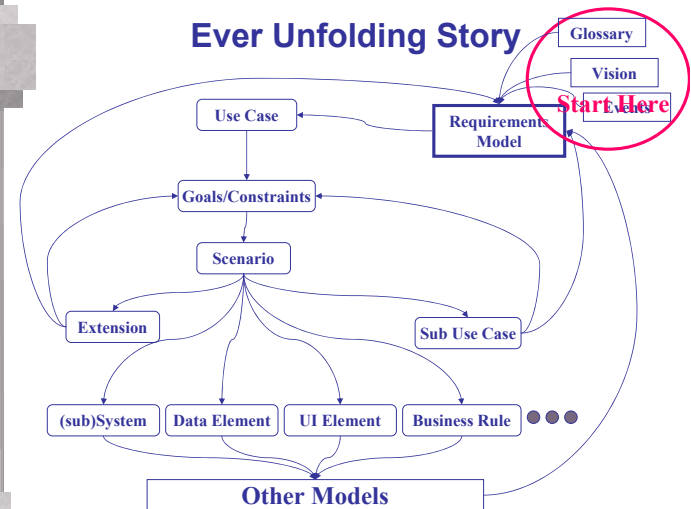
- Validate what we “know”
- Encapsulate what we don’t know by labeling it with an “Intention Revealing Name”
- Investigate, model, etc, these “new things”
- Repeat...



Main Difficulties (analysis paralysis)

- Digging too deep too fast
- Not having right people to validate

Ever Unfolding Story



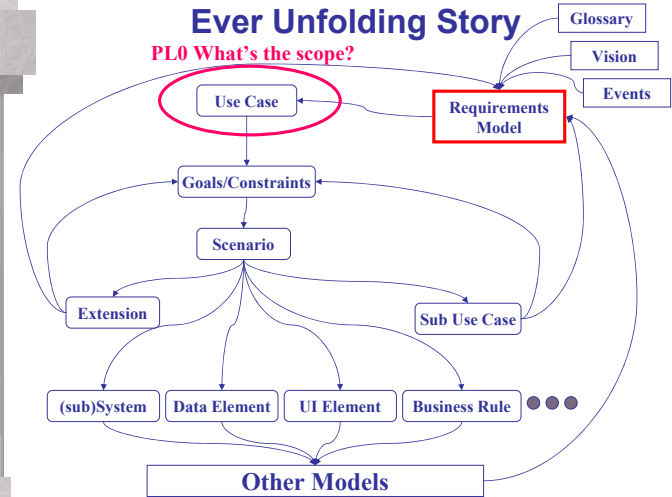
Samples

<i>For</i>	Small Consulting Group
<i>Who</i>	Is using paper for everything
<i>The SCG Application</i>	<i>Is a web-based tool</i>
<i>That</i>	Handles payroll, billing, and scheduling of Consultants
<i>Unlike</i>	Anything that we can find...
<i>Our Product</i>	Will enable us to practice what we preach

Event	Response
Course Completed	Invoice Client
Client asks for Course	Schedule course
End of Day	Employee fills our Timesheet
Payment Received from Client	Payment logged
Employee needs time off	Schedule Vacation Time

Ever Unfolding Story

PL0 What's the scope?



PL0: Who Wants What

Use Case: Withdraw Cash from ATM

Context of Use: ATM Customer wishes to withdraw money from ATM while executing the *Use ATM Machine* Use Case.

System: ATM Machine

Level: System (Sea Level, User Goal)

Primary Actor: ATM Customer

Stakeholders and interests:

ATM Customer -- wants cash returned and ATM Card returned. Wants a receipt. Wants to have appropriate amount of money deducted from account. Wants to pay as little fee as possible.

Bank Manager -- wants to have appropriate amount of money deducted from account. Wants as much fee as possible.

Bank Regulator -- wants to make sure all regulations are complied with

Precondition: ATM Customer is logged on

Postconditions:

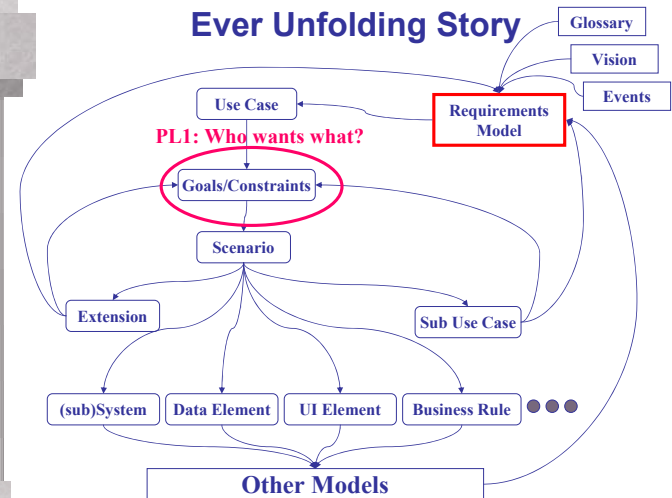
Success: Money and receipt dispensed to ATM Customer. ATM Card returned. Customer Account debited appropriate amount of money. Appropriate fees collected. All banking regulations adhered to.

Minimal: Transaction logged to completion or point of failure

Trigger: ATM Customer selects "Withdraw Cash" option at main menu

Ever Unfolding Story

PL1: Who wants what?



PL1: Who Wants What?

Use Case: Withdraw Cash from ATM

Context of Use: ATM Customer wishes to withdraw money from ATM while executing the *Use ATM Machine* Use Case.

System: ATM Machine

Level: System (Sea Level, User Goal)

Primary Actor: ATM Customer

Stakeholders and Interests:

ATM Customer -- wants cash returned and ATM Card returned. Wants a receipt. Wants to have appropriate amount of money deducted from account. Wants to pay as little fee as possible.

Bank Manager -- wants to have appropriate amount of money deducted from account. Wants as much fee as possible.

Bank Regulator -- wants to make sure all regulations are complied with

Precondition: ATM Customer is logged on

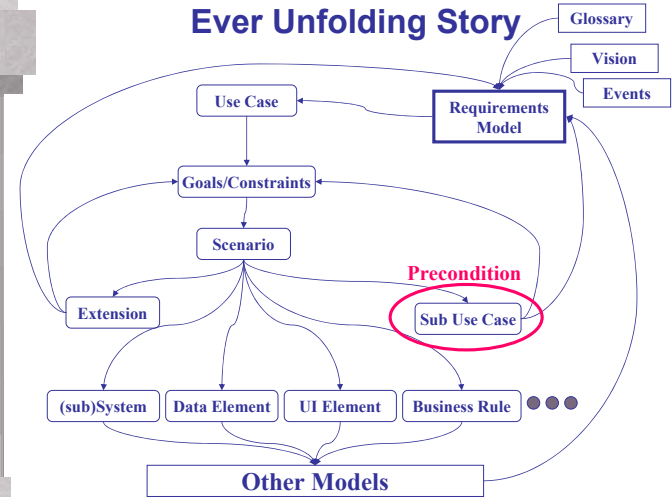
Postconditions:

Success: Money and receipt dispensed to ATM Customer. ATM Card returned. Customer Account debited appropriate amount of money. Appropriate fees collected. All banking regulations adhered to.

Minimal: Transaction logged to completion or point of failure

Trigger: ATM Customer selects "Withdraw Cash" option at main menu

Ever Unfolding Story



Documenting Preconditions in the Use Case

Use Case: Withdraw Cash from ATM

Context of Use: ATM Customer wishes to withdraw money from ATM while executing the *Use ATM Machine* Use Case.

System: ATM Machine

Level: System (Sea Level, User Goal)

Primary Actor: ATM Customer

Stakeholders and Interests:

ATM Customer -- wants cash returned and ATM Card returned. Wants a receipt. Wants to have appropriate amount of money deducted from account. Wants to pay as little fee as possible.

Bank Manager -- wants to have appropriate amount of money deducted from account. Wants as much fee as possible.

Bank Regulator -- wants to make sure all regulations are complied with

Precondition: ATM Customer is logged on

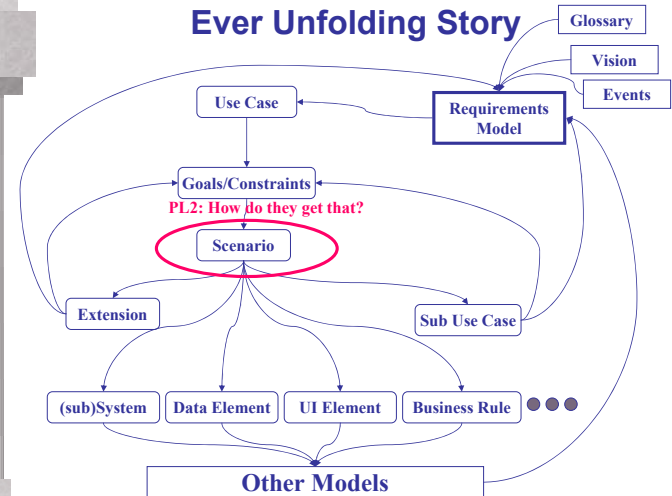
Postconditions:

Success: Money and receipt dispensed to ATM Customer. ATM Card returned. Customer Account debited appropriate amount of money. Appropriate fees collected. All banking regulations adhered to.

Minimal: Transaction logged to completion or point of failure

Trigger: ATM Customer selects "Withdraw Cash" option at main menu

Ever Unfolding Story



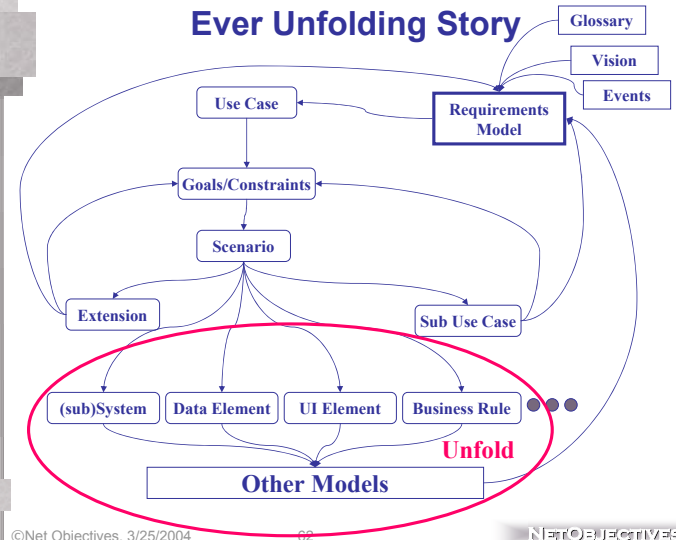
Use Case: Withdraw Cash from ATM

Main Success Scenario:

1. The ATM Customer selects an account to withdraw from, and enters an amount to withdraw, in multiples of \$20.
2. The ATM Machine verifies that there is enough money to dispense.
3. The ATM Machine notifies the Main Banking System of the Customer Account and amount being withdrawn, and the Main Banking System returns the new balance and (if appropriate) the fee amount.
4. The ATM Machine dispenses the cash and updates the receipt.

*The ATM Machine logs each step of this transaction.

Ever Unfolding Story



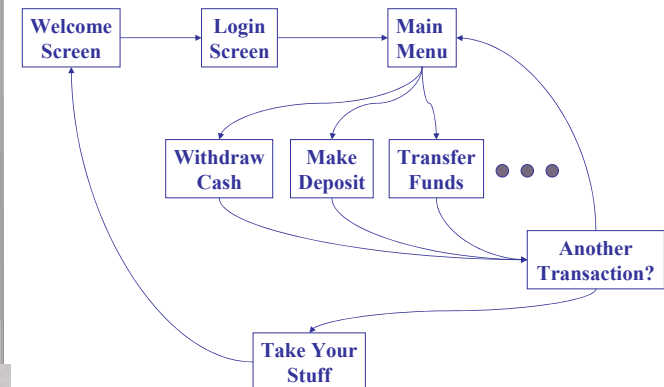
Use Case: Withdraw Cash from ATM

Main Success Scenario:

1. The ATMMachine displays the WithdrawCashScreen
2. The ATM Customer selects the WithdrawalAccount, from, enters the WithdrawalAmount, in multiples of \$20.
3. The ATMMachine verifies that there is enough money to dispense in its CashDispenser
4. The ATMMachine notifies the MainBankingSystem of the CustomerAccount and WithdrawalAmount, and the MainBankingSystem returns the new AccountBalance and the AssessedFee amount (if any)
4. The ATMMachine dispenses the WithdrawalAmount and updates the TransactionReceipt

*The ATM Machine logs each step of this transaction.

Story Board for ATM Machine



Interface For Login

Please Enter PIN: XXXX

Then Click Here →

Old-Style Kiosk

Please Enter Pin

XXXX

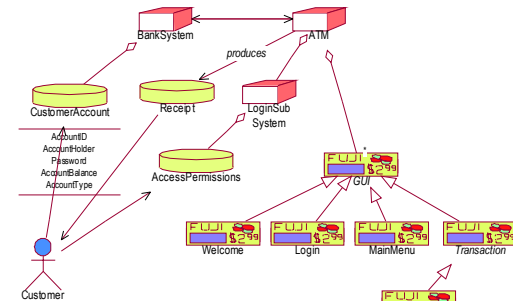
Enter

Touch-Screen Kiosk

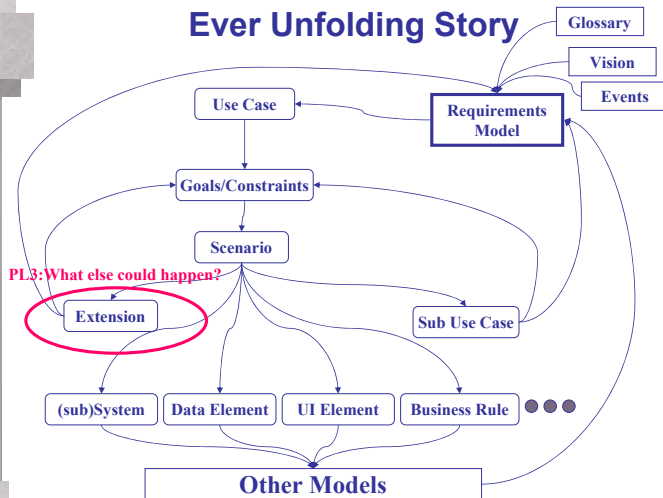
Logical Model w/Attributes

This is the UML's version of a logical model

- Don't worry about the "arrowheads"
- Just note that this is telling a story...



Ever Unfolding Story



Handling Extension Conditions

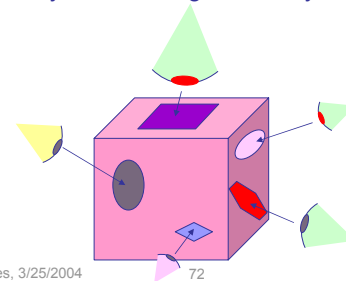
- Extension Conditions result in more scenarios in order to "solve" the problems caused by the extension
- We must determine if this problem is worth solving
 - The essence of agility, determining where to allocate our resources to increase robustness...
- If we decide to solve this problem...
 - We do the unfolding of those scenarios as we did for the main success scenario
 - Thus showing the inherent fractal nature of requirements, and how the Ever-Unfolding Story mirrors this nature

Needs/Wants

- 70

- ✓ Vision
- ✓ Glossary
- ✓ Event List
- ✓ Use Case
- ✓ Activity Diagrams
- ✓ StoryBoard
- ✓ UI Mockup
- ✓ Logical Data Diagram
- ✓ Business Rules
- ✓ Deployment Diagram

- The System is the System...
- Each Model views the System with a different focus
 - What is relevant, what is irrelevant?
- The hope is that with enough of the “right” models, you can actually “see” enough of the system to understand it...



Please Fill Out Evaluations



Conference Style Seminars For Your Organization

- We are happy to offer any of these seminars to:
 - your user group, if we are in your area and there will be sufficient attendance, at no charge
 - to your company for a modest fee.
- Please contact Mike Shalloway at mike.shalloway@netobjectives.com or call 404-593-8375 if you are interested in us providing a seminar for your user group or company.
- Many of these talks have been presented at conferences such as SD Expo, OOPSLA, and OOP.
- (All seminars are 90 minutes, unless noted otherwise.)
- **Agile Processes**
 - Comparing RUP, XP, and Scrum: Mixing a Process Cocktail for Your Team
 - Agility and Ceremony: How Can They Co-Exist?
 - Transitioning To Agile
 - The Need for Agility
- **Analysis**
 - Introduction to Use Cases
 - Use Cases and the Ever-Unfolding Story
- **Agile Coding Practices**
 - Pattern Oriented Development: Design Patterns From Analysis To Implementation
 - Effective Coding Practices
 - Emergent Design: Design Patterns and Refactoring for Agile Development
 - Unit Testing For Emergent Design
- **C#**
 - C# for Java and C++ Developers
 - Effective C#
- **XML**
 - XML Data Binding With Castor
 - XSLT - Step By Step
 - The JDOM Alternative For XML Parsing

Net Objectives – Who We Are

- **Mission:** Guiding and empowering our customers in their software development efforts via courses and coaching.
- **Vision:** Effective and efficient software development without suffering.
- We provide training, mentoring and consulting for all phases of object-oriented software development.
- We assist companies transitioning to agile and object-oriented development by providing mentoring throughout the entire development process.

Use Cases and Analysis

- **Use Case Based Requirements Analysis (UCBRA).** Use Cases are the best tool known for capturing and documenting the functional requirements for a system. Unfortunately, trying to capture a system's use cases all at once often leads to a severe case of "analysis paralysis". The Ever-Unfolding Story is an incremental technique for developing use cases and refining them into more detailed requirements. Because it's incremental, the Ever-Unfolding Story allows for frequent validation of the requirements, and supports both artifact-driven and agile development processes.
- **Agile Use Cases (AUC).** Agile software development is based on a simple concept: work on the most important things first, and iteratively look for what is 'now' the most important thing. By doing this a development team is able to adapt to changes as they occur, and maximize effectiveness of the development effort. We use the term "Agile Use Cases" to refer to an incremental technique for iteratively developing Use Cases and refining them into software requirements using the Ever-Unfolding Story. Agile Use Cases provide for frequent validation of the requirements, thus supporting agile development.
- These courses are similar in content, but with different focus: UCBRA is for larger, more traditional, projects; while AUC is designed to support agile developments. Each of these 3-day courses consists of lecture and hands-on exercises, and the use case portion is largely based on Alistair Cockburn's book "Writing Effective Use Cases" - winner of the Jolt Productivity Award for 2001.

Agile Development and OOA/UML

- **Agile Development Best Practices.** In simple terms, an Agile Project is one that is predicated on making sure it is always doing the right thing, not merely following a plan that has since gone out of date. The cornerstone of this approach is getting and adapting to feedback as the project progresses. Most projects can't do this, so they fall further behind and either fail or provide inferior products. Changes are of many types, but the most common (and important) changes are to the system's requirements. This course analyzes what it means to be an agile project, and provides a number of best practices that provide and/or enhance agility. Different agile practices (including RUP, XP and Scrum) are discussed.
- **Object-Oriented Analysis.** The word "analysis" means many things when discussing software development. In this two-day course you will learn about three different kinds of analysis: Domain Analysis, Requirements Analysis, and Architectural Analysis. The course consists of lecture and many hands-on exercises; you will learn a number of different modeling techniques, and use many common Unified Modeling Language (UML) diagrams. Throughout the course the focus is on the Analyst's job - providing information that will give other developers a "fighting chance" to develop the correct software.

Object-Oriented Analysis and Design

- **Effective Object-Oriented Analysis and Design.** This course is taught in C++, C#, and/or Java. C++, C#, and Java are powerful programming languages. Their true value emerges only if they are applied with good object-oriented practices. Without these, they can be a difficult language to maintain and extend. Although many people understand the basics of object-oriented programming (polymorphism, interfaces, encapsulation, ...) they don't understand how to use these effectively. This three-day course starts with a better way to do analysis than is traditionally taught. It follows up with 14 practices that both enable better object-oriented coding as well as create a better understanding of object-oriented design. These practices come from both design patterns and new agile coding methods.
- This course is about 40% lab.

Design Patterns and Refactoring

- **Design Patterns Explained: A New Perspective on Object-Oriented Design.** This course goes beyond merely teaching several design patterns. It also teaches the principles and strategies that make design patterns good designs. This enables students to use these advanced design techniques in their problems whether design patterns are even present. After teaching several patterns and the principles underneath them, the course goes further by showing how patterns can work together to create robust, flexible, maintainable designs.
- **Refactoring, Unit Testing, and Test-Driven Development.** The practice of Agile Software Development requires, among other things, a high degree of flexibility in the coding process. As we get feedback from clients, stakeholders, and end users, we want to be able to evolve our design and functionality to meet their needs and expectations. This implies an incremental process, with frequent (almost constant) change to the code we're working on. Refactoring, the discipline of changing code without harming it, is an essential technique to enable this process. Unit testing, which ensures that a given change has not caused an unforeseen ripple effect in the system, is another.

Java: On-site and Instructor Led Web-Based Training

- **Object-Oriented Programming: Java Edition.** Java is a powerful programming language, but its true value emerges only when a solid understanding of Object Oriented Programming techniques are applied. This two-day course focuses on the essential concepts of OO: Encapsulation, Inheritance, and Polymorphism, and includes practical examples and exercises using Java code. Emphasis is placed on implementation issues such as strong cohesion, loose coupling, and eliminating redundancy. Best practices, as illustrated by common design patterns are also discussed.
- **On-line/instructor led Java Training.** This is our on-line equivalent of our 5-day Java training. This is not a self-study program where you get e-mail answers to your question. This is a high-touch training program where you work in frequent (albeit remote) contact with both the instructor and your student community. The core of the Java language is taught as well as some SWING and many of the core Java classes. It requires about 4 hours a week from each participant and takes place over 12 weeks.

C#: On-site and Instructor Led Web-Based Training

- **C# for C++ and Java Developers.** C# is the flagship language for .NET, and despite what many have suggested, it is neither Java with enhanced syntax, nor is it C++ with better manners. C# is a new language, with many new syntactic elements. Also, programming in .NET requires an understanding of the framework and the development process it is designed to support. This 1-day course is intended to elucidate the C# language in terms of syntax, process, and some early-adopter best practices, making the transition for Java and C++ developers as smooth as possible.
- **On-line/Instructor Led C# Training.** This course teaches the basics of the C# language as well as many of the standard C# classes that support the language. It is equivalent to a 5-day C# training class. It requires about 4 hours a week from each participant and takes place over 12 weeks.

Object-Oriented Programming in .NET

- **Object-Oriented Programming: C# Edition.** The emergence of .Net for distributed computing has brought along with it a new flagship language: C# (C-Sharp). Those who are familiar with C, C++, and Java will find the syntax of C# relatively easy to absorb, but using the language properly will require a firm grounding in OO concepts and practices. This two-day course focuses on the essential concepts of OO: Encapsulation, Inheritance, and Polymorphism, and includes practical examples and exercises using C# code. Emphasis is placed on implementation issues such as strong cohesion, loose coupling, and eliminating redundancy. Best practices, as illustrated by common design patterns are also discussed.
- **Object-Oriented Programming: VB.NET Edition.** Visual Basic (VB) has undergone a major transformation with the new version for .Net. What was once an object-based, procedural language has now become an Object Oriented language akin to C++, Java, and C#. While the primary aspects of its syntax has not changed, this shift to a true Object Orientation will require those with an investment in VB knowledge to learn new concepts such as inheritance, polymorphism, designing to interfaces, and object composition. This 2-day course will take the experienced VB 5.0 or VB 6.0 developer and teach them the OO concepts they need to move to VB.Net.

C++ and XML

- **Object-Oriented Programming: C++ Edition.** C++ is a powerful programming language, but its true value emerges only when a solid understanding of Object Oriented Programming techniques are applied. This two-day course focuses on the essential concepts of OO: Encapsulation, Inheritance, and Polymorphism, and includes practical examples and exercises using C++ code. Emphasis is placed on implementation issues such as strong cohesion, loose coupling, and eliminating redundancy. Best practices, as illustrated by common design patterns are also discussed.
- **XML and Its Family of Technologies.** XML is a powerful new standard for data modeling and communication, controlling presentation, and inter/intra-process communication. As XML is a cross-platform tool, it is a natural fit for the Java programming language, which is also operating system independent. This course introduces the student to XML and its primary supportive technologies such as SAX/DOM, DTD/Schema, JDOM, and XSLT, with an emphasis on the motivations behind these tools and the way they interrelate. The instructor will then provide detailed, practical examples of how these tools can be used in a Java-based project.